Politecnico di Torino

Master's Degree in Computer Engineering

Migrating Legacy Services to a Cloud-Native Architecture: A Case Study on Outsight Cloud

Supervisor: Prof. Fulvio Giovanni Risso

Candidate: Giovanni Russo
Matricola: s343424

Academic Year 2025–2026

Abstract

Legacy systems often evolve through years of incremental development, resulting in architectures that become difficult to scale, maintain, and modernize. This thesis investigates the migration of Outsight Cloud, a production platform for managing LiDAR-related workflows and assets, from a legacy service-oriented architecture toward a cloud-native environment on AWS.

The original platform was built around external platforms such as Airtable and manually managed operational processes, which progressively introduced scalability, consistency, and maintenance issues.

The thesis proposes and evaluates an incremental migration strategy based on Infrastructure as Code, managed cloud services, and progressive coexistence between legacy and target components. Particular attention is given to the transition from Airtable to PostgreSQL, the adoption of Terraform-driven infrastructure provisioning, and the introduction of validation mechanisms designed to reduce migration risk while preserving service continuity.

The proposed approach combines architectural decision-making with technical and operational evaluation criteria, including reproducibility, maintainability, deployment reliability, observability, and cost sustainability. To reduce migration risk and preserve system continuity, the project adopts incremental validation practices, automated consistency checks, and reproducible deployment workflows across isolated environments.

The resulting architecture demonstrates how cloud-native modernization can be achieved incrementally in a real industrial context through controlled evolution, continuous validation, and risk-aware design practices. Beyond the specific case study, the thesis provides a practical framework for organizations facing similar legacy-to-cloud migration challenges.

Acknowledgements

I would like to express my sincere gratitude to Outsight for providing me with the opportunity to complete this internship and work on challenging and meaningful projects. I am particularly thankful to my supervisor and the entire development team for their guidance, support, and patience throughout this journey.

I would also like to thank the Politecnico di Torino for the academic foundation that made this work possible, and my thesis advisor for valuable feedback and encouragement.

Finally, I am deeply grateful to my family and friends for their unwavering support during my studies.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Industrial Context and Legacy Constraints

In recent years, cloud computing has significantly transformed the way software systems are designed, deployed, and maintained. Modern cloud platforms provide scalable infrastructure, managed services, automated deployment capabilities, and operational flexibility that are difficult to achieve with traditional self-managed environments. As a consequence, many organizations are progressively modernizing legacy applications to adopt cloud-native architectures.

However, migrating an existing production platform to the cloud is rarely a straightforward process. Legacy systems are typically the result of years of incremental development shaped by evolving business requirements, operational constraints, and technological decisions. Over time, these systems often accumulate architectural dependencies and operational practices that complicate scalability, maintainability, and long-term evolution.

This thesis analyzes the migration of Outsight Cloud (OC), a platform used to manage LiDAR-related workflows, simulations, organizations, sites, and associated resources. Before the migration effort, the platform relied on a Docker Compose-based architecture integrated with several external services, including Airtable for core data management.

The original architecture enabled rapid development and operational flexibility during the early stages of the product lifecycle. Airtable simplified data management and allowed non-technical stakeholders to interact directly with operational

data through a graphical interface, while externally managed services reduced initial infrastructure complexity and accelerated feature delivery.

As the platform evolved, however, several limitations progressively emerged. Core business logic became increasingly dependent on external platforms and semi-structured data representations, making the system harder to maintain and evolve. At the same time, operational workflows required growing amounts of manual intervention, while scalability and infrastructure management became more difficult as platform usage and integrations expanded.

The migration effort described in this thesis was therefore motivated not only by infrastructure modernization goals, but also by the need to redesign the architectural foundations of the platform in a more scalable, maintainable, and operationally sustainable way.

1.2 Problem Statement

The engineering problem addressed in this work concerns the migration of Outsight Cloud from a legacy architecture toward a cloud-native environment under real industrial and operational constraints.

Although the original platform architecture supported fast development and iterative product evolution, over time it introduced technical and operational limitations that reduced architectural flexibility and increased maintenance complexity. In particular, the platform became strongly dependent on external services and platform-specific integrations, especially Airtable, which was deeply embedded in application workflows and operational processes.

This dependency introduced several challenges. The absence of a fully relational data model limited consistency guarantees and complicated the management of relationships between entities. Operational procedures increasingly relied on manual or partially automated workflows, generating inefficiencies and increasing the risk of human error. In parallel, the original deployment model became progressively harder to scale and maintain as infrastructure requirements, integrations, and operational complexity continued to grow.

To address these limitations, the company initiated a cloudification process aimed at redesigning the platform architecture and migrating services to AWS. The

migration objective extended beyond simple workload relocation and required a broader architectural transformation involving infrastructure redesign, adoption of Infrastructure as Code practices, migration from Airtable to PostgreSQL, introduction of managed cloud services, and modernization of deployment and operational workflows.

A central challenge of the migration was preserving service continuity while legacy and target components coexisted during transition phases. For this reason, the migration strategy was designed as an incremental process supported by validation workflows, reproducible infrastructure management, and controlled coexistence mechanisms capable of reducing technical and operational risk.

1.3 Thesis Structure

This thesis is organized as follows:

Chapter 2 introduces the conceptual and related-work foundations on cloud-native modernization, migration patterns, and IAM evolution in legacy systems.

Chapter 3 describes the case-study baseline (as-is architecture), including technical debt and cost-related migration drivers.

Chapter 4 presents the migration methodology and decision framework, including candidate options and selection rationale.

Chapter 5 defines the target architecture (to-be), its main components, and design trade-offs.

Chapter 6 details the incremental migration implementation foundations, with focus on IaC, data migration tooling, and validation pipeline.

Chapter 7 discusses the evaluation approach and current evidence regarding migration feasibility, quality, and impact.

Chapter 8 concludes the thesis by summarizing contributions and outlining future work.

Chapter 2

Background and Related Work

This chapter provides the conceptual background required to frame the migration problem and positions the thesis with respect to related work.

2.1 Cloud-Native Principles for Legacy Modernization

Cloud-native modernization is not limited to container adoption or workload relocation. In migration contexts, it also requires reproducible infrastructure, explicit service boundaries, deployment automation, operational observability, and validation mechanisms capable of supporting incremental system evolution. These principles become particularly important when legacy and modern components must coexist during transition phases.

Modern cloud-native systems are typically designed around automation-oriented practices and managed services that simplify infrastructure operations and improve scalability. However, migrating a legacy platform toward such architectures requires careful consideration of operational continuity, compatibility constraints, and long-term maintainability.

2.1.1 Incremental Delivery and Operability

A recurring principle in modernization literature is the preference for incremental delivery over disruptive big-bang replacement. Incremental migration reduces operational risk, enables earlier feedback cycles, and allows validation activities to be performed progressively throughout the transition process.

To be effective, incremental delivery requires infrastructure automation, environment consistency, reproducible deployment workflows, and testable interfaces between legacy and target components. In industrial systems, these practices are essential for preserving operational continuity while introducing architectural changes.

2.1.2 Infrastructure as Code

Infrastructure as Code (IaC) is a key enabler for cloud-native modernization because it transforms infrastructure configuration into versioned and reproducible artifacts. Instead of relying on manual provisioning procedures, infrastructure resources are defined declaratively and managed through automated workflows.

This approach improves traceability, reproducibility, reviewability, and consistency across development and production environments. In migration projects, IaC also simplifies environment replication, deployment governance, and rollback management, reducing operational risks during infrastructure evolution.

2.2 Migration Strategies and Patterns

Cloud migration projects can follow different strategies depending on system complexity, operational requirements, and modernization goals. Common approaches include rehosting, replatforming, refactoring, and selective rebuilding. In practice, enterprise migration projects often combine multiple strategies across subsystems according to their coupling level, criticality, migration effort, and risk exposure.

2.2.1 Cloud Migration Fundamentals

Cloud migration is the process of moving applications, data, and infrastructure resources from traditional environments to cloud-based platforms [1]. In practice, this

transition is executed progressively through staged rollout strategies and validation activities designed to minimize disruption.

Historically, many organizations operated software systems on self-managed infrastructures hosted in local data centers or virtualized environments. While these architectures provided direct control over resources, they also introduced significant operational complexity related to infrastructure maintenance, capacity planning, backup management, monitoring, and service availability.

Cloud migration should therefore not be interpreted as simple workload relocation. In many cases, migration also requires architectural redesign decisions intended to improve scalability, maintainability, automation capabilities, and long-term operational efficiency.

Reference guidance highlights several recurring migration drivers and benefits [1], including cost optimization, elastic scalability, managed operational services, improved infrastructure reliability, and access to modern distributed delivery models.

Cloud migration strategies are generally classified into three major categories [1]: rehosting, which moves workloads with minimal architectural changes; replatforming, which introduces selective adaptations to better exploit cloud capabilities; and refactoring, which redesigns parts of the system to align with cloud-native principles.

Migration programs are often organized into assessment, preparation, and migration phases [1]. These phases support technical discovery, environment preparation, pilot execution, and incremental rollout activities.

Typical migration challenges include legacy technical complexity, execution constraints during coexistence phases, and operational adaptation to new infrastructure models. For this reason, successful migration projects require not only technical redesign, but also controlled validation and incremental transition strategies.

In the Outsight Cloud case, migration extended beyond infrastructure relocation. The project required redesign of deployment workflows, replacement of legacy dependencies, migration from Airtable to PostgreSQL, and adoption of Infrastructure as Code practices to support long-term maintainability and scalability.

2.2.2 Coexistence as a First-Class Requirement

For production systems that cannot tolerate service interruption, coexistence between legacy and target components becomes a critical architectural requirement. During migration phases, both systems frequently need to operate simultaneously while compatibility and behavioral consistency are progressively validated.

This coexistence model introduces additional engineering complexity, including contract compatibility, dual-backend validation, synchronization workflows, rollback capabilities, and progressive switchover mechanisms. As a consequence, migration validation becomes an integral part of the system architecture rather than a secondary operational activity.

2.2.3 Decision-Driven Migration

Related work frequently describes migration patterns from a purely technological perspective, focusing primarily on target architectures and adopted cloud services. In industrial contexts, however, migration decisions must also consider operational constraints, organizational impact, recurring costs, maintainability, and migration risk.

For this reason, migration strategies should be supported by explicit decision frameworks capable of combining technical and economic rationale. This aspect is particularly relevant in large-scale modernization projects where architectural choices affect both infrastructure sustainability and long-term operational governance.

2.3 IAM Modernization in Legacy Systems

Identity and Access Management (IAM) modernization is a recurrent challenge in legacy platforms. Historically, many systems implemented authentication and authorization logic directly inside application workflows and persistence layers, generating tightly coupled identity-management models that become difficult to evolve and maintain over time.

Modern cloud-native architectures instead rely on standardized protocols and dedicated identity services to improve interoperability, scalability, and security.

2.3.1 From Application-Coupled Identity to Standard Protocols

Legacy systems frequently embed authentication assumptions directly into business logic and data models. Migrating toward standards-based IAM therefore requires decoupling identity management from application-specific implementations and redefining authorization boundaries.

Protocols such as OAuth 2.0 and OpenID Connect (OIDC) provide stronger interoperability and security guarantees than ad-hoc authentication approaches. However, introducing standardized IAM mechanisms into existing systems also requires compatibility validation with existing workflows, clients, and operational procedures.

2.3.2 Open-Source and Managed IAM Trade-offs

The choice between open-source IAM solutions and managed identity platforms affects operational complexity, integration flexibility, infrastructure ownership, and long-term maintenance costs.

Managed IAM services simplify operational governance by reducing infrastructure-management responsibilities and providing built-in support for authentication workflows, user lifecycle management, and security integration. Open-source solutions, on the other hand, generally provide greater customization flexibility and infrastructure control at the cost of increased operational responsibility.

In migration projects, these trade-offs also influence how progressively existing services can adopt modern authentication flows while preserving backward compatibility.

2.4 Limitations in Existing Approaches

The literature provides strong conceptual foundations on cloud migration patterns and IAM technologies. However, industrial guidance often remains fragmented across separate technical dimensions.

2.4.1 Fragmented Treatment of Technical and Economic Criteria

Technical architecture decisions are frequently discussed without detailed consideration of operational costs, infrastructure governance, or long-term maintenance overhead. In practice, however, recurring infrastructure costs and dependency-management considerations are often decisive factors in enterprise migration strategies.

As a consequence, migration projects require evaluation frameworks capable of combining scalability, maintainability, operational sustainability, and economic impact within a unified decision process.

2.4.2 Limited Evidence on Incremental Validation

Many migration case studies focus primarily on final target architectures while providing limited evidence regarding coexistence management and intermediate validation mechanisms.

In industrial environments, however, migration reliability often depends on validation practices such as dual-backend testing, data-consistency verification, environment-separated deployment workflows, and incremental rollout strategies. These aspects are essential for reducing migration risk during long transition phases.

2.5 Positioning of This Thesis

This thesis addresses the above limitations through a decision-oriented migration case study focused on incremental architectural evolution under real industrial constraints.

The contribution of the work is not limited to the description of a target AWS architecture. Instead, the thesis emphasizes the reasoning process that connects baseline system limitations, migration decisions, infrastructure automation, coexistence management, and validation workflows.

Particular attention is given to Infrastructure as Code adoption, migration reproducibility, controlled coexistence between legacy and target systems, and validation-oriented migration practices designed to reduce technical and operational risk.

Although the work is centered on Outsight Cloud, the migration principles and engineering practices discussed throughout the thesis are applicable to broader legacy modernization scenarios involving progressive cloud-native adoption.

Chapter 3

Case Study Baseline (As-Is System)

This chapter describes the initial state of the Outsight Cloud platform before the migration strategy was formalized. The objective is to establish the architectural and operational baseline against which migration decisions and modernization efforts are evaluated.

3.1 Current Architecture and Data Flow

Before the migration to AWS, Outsight Cloud relied on a containerized architecture designed to support rapid feature delivery and operational flexibility during the early stages of product development.

The platform managed LiDAR-related workflows and assets through a web application and backend API stack composed of several interconnected services. The frontend was implemented as a Vue 3 Single Page Application (SPA), while backend logic was handled by a Node.js/Express API service. Session management relied on Redis, and the overall deployment was orchestrated through Docker Compose with Traefik acting as reverse proxy and HTTPS entry point.

The platform also integrated multiple external services responsible for storage, documentation, notifications, and operational workflows. File storage relied on an S3-compatible object storage service, while technical documentation was hosted through GitBook. Notification-related functionalities depended on integrations with

Slack and Customer.io.

A central characteristic of the original architecture was the strong dependence on Airtable as the primary operational datastore. Core business entities such as users, organizations, sites, simulations, permissions, and role assignments were all managed through Airtable-based workflows.

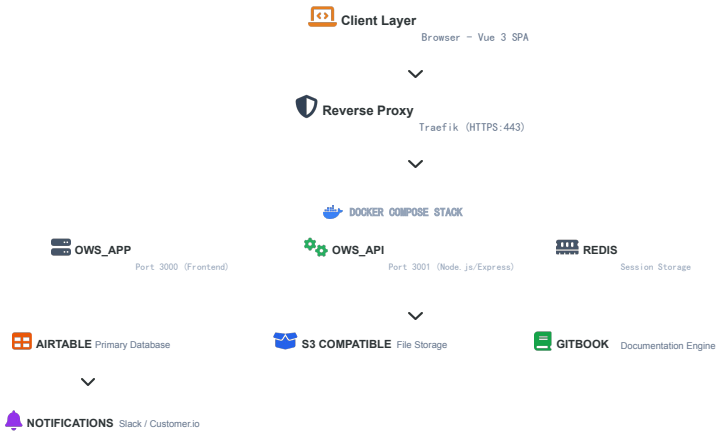

Figure 3.1. Outsight Cloud baseline architecture before the migration.

Although this architecture enabled fast iteration and reduced initial infrastructure complexity, over time it introduced increasing dependency concentration around externally managed services and operationally coupled workflows.

3.2 Airtable as Primary Data Storage

Before the migration to PostgreSQL, Airtable represented the central data-management component of Outsight Cloud. Its role extended beyond traditional data persistence, functioning simultaneously as operational datastore, workflow-management layer, and integration hub for several internal processes.

Airtable stored most of the platform's core business entities, including users, organizations, sites, simulations, permissions, and role assignments. In addition, it

supported operational forms and triggered integrations with external systems such as Slack-based notification workflows.

Unlike traditional relational database systems, Airtable organizes information through linked records and lookup fields instead of explicit relational schemas and foreign-key constraints. During the early phases of the project, this model provided significant flexibility and accelerated development activities. Non-technical stakeholders could directly inspect and modify operational data through Airtable's graphical interface, simplifying collaboration between technical and operational teams.

This flexibility was particularly useful during rapid product evolution phases, where requirements changed frequently and development speed was prioritized over strict schema governance.

However, as the platform evolved and the volume of managed data increased, several structural limitations progressively emerged.

3.3 Limitations of Airtable

As Outsight Cloud became more complex, the limitations of using Airtable as primary backend infrastructure became increasingly evident. The issues affected not only data consistency and backend maintainability, but also operational workflows, scalability, and long-term architectural evolution.

3.3.1 Data Modeling and Consistency

Relationships between entities such as organizations, sites, simulations, and permissions were represented through linked records and lookup fields rather than through explicit relational constraints.

As a consequence, consistency guarantees depended heavily on application-side logic instead of database-level enforcement mechanisms. Backend services frequently needed to normalize and reinterpret Airtable responses in order to maintain coherent internal representations.

In several cases, lookup fields exposed heterogeneous data structures depending on the access context. The same logical information could appear either as scalar values or arrays, increasing backend complexity and complicating synchronization and mapping logic.

This absence of strict schema enforcement progressively increased technical debt and reduced maintainability as the number of entities and relationships expanded.

3.3.2 Limited Relational Capabilities

Although Airtable supports basic linked-record relationships, it does not provide the full relational capabilities of SQL database systems such as PostgreSQL.

Advanced joins, indexing strategies, transactional consistency, complex filtering operations, and optimized querying mechanisms were significantly more limited. As business logic became more sophisticated, these limitations increasingly affected backend implementation complexity and query efficiency.

The platform therefore required growing amounts of application-side logic to compensate for missing relational features that would normally be handled directly by a relational DBMS.

3.3.3 Operational Inefficiency

Several operational workflows relied on manual interactions through Airtable forms and operational interfaces.

User onboarding and account-management procedures, for example, often required manual intervention from internal teams. This approach slowed operational processes, increased dependency on human intervention, and raised the probability of operational errors.

As the number of users, organizations, and managed simulations increased, these manual procedures became progressively harder to scale efficiently.

3.3.4 Scalability, Coupling, and Cost

Over time, Airtable evolved from a simple operational datastore into a critical architectural dependency tightly coupled with application workflows and operational procedures.

Business logic progressively became dependent on Airtable-specific concepts and APIs, reducing architectural flexibility and complicating future system evolution. This dependency concentration increased technical debt and limited the maintainability of the platform.

The economic impact also became significant. Recurring operational costs associated with Airtable usage reached approximately EUR 1000 per month, creating additional pressure to adopt a more scalable and sustainable backend architecture.

3.4 Motivation for Migration to PostgreSQL

The migration from Airtable to PostgreSQL was initiated to address the architectural, operational, and economic limitations identified in the previous sections.

The objective was not simply to replace one storage technology with another, but to progressively transition from a semi-structured operational backend toward a maintainable and scalable relational architecture capable of supporting long-term platform evolution.

PostgreSQL introduced several advantages compared to the previous data-management model. Explicit relational schemas based on primary and foreign keys enabled stronger consistency guarantees and improved integrity enforcement. Advanced querying capabilities, transactional support, and indexing mechanisms also simplified backend implementation and improved data-management efficiency.

From an architectural perspective, the migration reduced dependence on Airtable-specific structures and allowed the platform to regain ownership of its data model and persistence logic. This improved maintainability and simplified future evolution of backend services.

The migration also aligned with the broader cloudification strategy of Outsight Cloud and the transition toward AWS-managed infrastructure and cloud-native operational practices.

For these reasons, PostgreSQL became a foundational component of the target architecture, supporting the broader modernization effort described in the following chapters.

Chapter 4

Migration Methodology and Decision Framework

This chapter formalizes the migration methodology used in this thesis. The emphasis is on how decisions were made, not only on which technologies were selected.

4.1 Migration Goals and Success Criteria

The migration goals were defined before implementation to avoid solution bias:

- Preserve operational continuity during transition.
- Reduce architectural coupling to legacy data and identity assumptions.
- Improve deployment reproducibility and environment governance.
- Provide explicit technical and economic justification for major choices.

From these goals, the following success criteria were derived:

- ability to run and validate critical flows during data-layer transition;
- traceable infrastructure definitions across environments;
- measurable reduction of dependency concentration on Airtable;
- migration process auditable through tests, scripts, and CI evidence.

4.2 Candidate Solutions Considered

The decision process considered multiple options for each migration dimension.

4.2.1 Infrastructure and Runtime Options

- Keep existing deployment style with incremental hardening.
- Move to managed cloud services with IaC governance.
- Adopt a full orchestration-first redesign from the beginning.

4.2.2 Alternative Cloud Platforms Considered

During the migration design phase, multiple cloud platform alternatives were considered before selecting the target environment. The comparison focused on scalability, availability of managed services, IaC support, ecosystem maturity, operational complexity, and integration with the existing technical baseline.

Table 4.1. Comparative assessment of cloud provider alternatives for Outsight Cloud migration

Criterion	**AWS**	**Azure**	**GCP**	**On-prem**
Managed services fit	High	Medium	Medium	Low
IaC and automation	High	Medium	Medium	Low
Operational complexity	Low to medium	Medium	Medium	High
Ecosystem and maturity	Very high	High	High	Variable
Migration continuity with current baseline	High	Medium-low	Medium-low	Low

Microsoft Azure

Microsoft Azure offers a broad enterprise ecosystem with managed services for compute, storage, databases, networking, and DevOps workflows. Functionally, it provides equivalents for many services used in this project, including:

- Azure Virtual Machines (similar role to EC2),
- Azure Kubernetes Service (AKS),
- Azure SQL Database,
- Azure Blob Storage,
- Azure DevOps.

Azure is particularly strong in Microsoft-centric enterprise contexts. However, for Outsight Cloud it introduced additional migration overhead because the existing deployment strategy and operational knowledge were already aligned with AWS-oriented patterns.

Google Cloud Platform (GCP)

Google Cloud Platform was also considered, especially due to its strengths in containerized workloads and Kubernetes-native operations. Relevant services included:

- Google Kubernetes Engine (GKE),
- Cloud SQL,
- Compute Engine,
- Cloud Storage.

GCP presents strong capabilities in analytics, machine learning, and managed Kubernetes. Nevertheless, AWS was preferred for this migration due to broader service coverage in the specific infrastructure management areas required by the project and stronger alignment with the existing cloudification path.

On-prem Infrastructure

Another option was to continue with self-managed virtual-machine infrastructure (or on-premises style operations). While this approach offers full control over resources and configurations, it significantly increases operational responsibilities:

- infrastructure maintenance,

- scalability management,
- backup and disaster recovery operations,
- monitoring and incident handling,
- high-availability design,
- security hardening and lifecycle management.

This option conflicted with the migration objective of reducing operational overhead through managed cloud services.

4.2.3 Data-Layer Migration Options

- Immediate cutover from Airtable to a relational database.
- Long-term coexistence with no formal migration endpoint.
- Incremental coexistence with progressive parity checks and controlled switchover.

4.2.4 Identity Evolution Options

- Preserve legacy authentication contracts and postpone IAM modernization.
- Introduce standards-based IAM with progressive integration in services.
- Full immediate rewrite of authentication and authorization boundaries.

4.3 Technical and Economic Evaluation Criteria

Each candidate solution was evaluated against a shared set of criteria.

4.3.1 Technical Criteria

- Backward compatibility with existing service behavior.
- Security posture and readiness for standards-based IAM.
- Deployment reproducibility across environments.

- Testability of migration steps and rollback readiness.
- Observability and troubleshooting support during transition.

4.4 Selected Strategy and Rationale

The selected strategy was an incremental migration centered on managed cloud services and reproducible infrastructure definitions:

- AWS-managed building blocks for runtime, routing, storage, and observability.
- Terraform as the infrastructure control plane.
- Progressive data migration path from Airtable to PostgreSQL.
- Validation-oriented coexistence between legacy and target data paths.

AWS was selected because it provided the most complete fit for the project constraints and goals:

- mature IaC integration through Terraform-based workflows,
- managed services required by the target architecture (including ECS, RDS, and S3),
- strong scalability and availability characteristics,
- robust DevOps integration for automated provisioning and updates,
- strong support for cloud-native architecture evolution,
- continuity with the broader company cloudification strategy,
- alignment with the company's commercial and strategic technology choices.

This strategy was selected because it balances risk, implementation effort, and long-term maintainability. It avoids disruptive full replacement while still creating a clear target-state trajectory.

4.5 Risk Management and Coexistence Plan

Risk management was embedded in the migration design:

- Keep migration steps independently verifiable.
- Separate development and production environment evolution.
- Use dual-backend test paths to detect behavioral divergence early.
- Define data validation artifacts to support switchover decisions.

The coexistence phase is treated as a controlled engineering stage, not as an informal transition period.

Chapter 5

Target Architecture and Solution Design (To-Be)

This chapter presents the target architecture resulting from the decision framework in Chapter 4. The architecture is described as a migration destination designed for incremental adoption.

5.1 Initial Architecture of Outsight Cloud

Before the migration to AWS, Outsight Cloud relied on a containerized architecture deployed through Docker Compose and integrated with multiple external services. This was the initial technical baseline of the platform.

The initial architecture was organized into four layers:

- client layer;
- reverse proxy layer;
- application layer;
- external services layer.

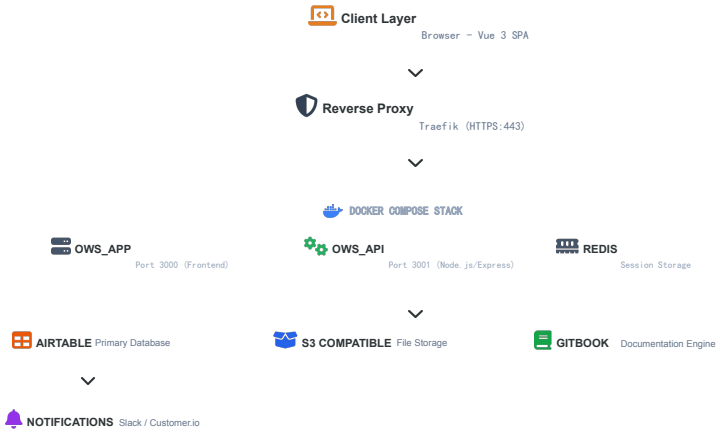

Figure 5.1. Initial architecture of Outsight Cloud before AWS migration.

In this setup, users accessed a Vue.js Single Page Application (SPA) through a web browser. Requests were routed by Traefik and forwarded to application services running in Docker containers. The backend integrated external systems including Airtable, S3-compatible storage, GitBook, Redis, and notification services.

5.1.1 Client Layer

Browser – Vue.js SPA

The frontend entry point was a browser-based SPA implemented in Vue.js. Its responsibilities included:

- rendering the user interface;
- managing user interactions;
- invoking backend APIs;
- displaying simulations, organizations, sites, and documentation.

5.1.2 Reverse Proxy Layer

Traefik Reverse Proxy

Traefik handled incoming HTTPS traffic and routed requests to internal services. Main responsibilities were HTTPS termination, request routing, container discovery, and traffic forwarding. The application was exposed externally through port 443, while internal service communication relied on Docker networking.

5.1.3 Application Layer

The application logic was deployed through a Docker Compose stack with three primary components.

Frontend Container

OWS_APP hosted the Vue frontend and exposed it on port 3000, delivering the web interface and client-side behavior.

Backend Container

OWS_API implemented backend logic with Node.js/Express and exposed APIs on port 3001. It orchestrated users, organizations, sites, simulations, file access, notifications, and external integrations.

Redis

Redis was primarily used as session storage and temporary state support, reducing repeated access to external systems and improving responsiveness.

5.1.4 External Services Layer

Airtable

Airtable was the primary datastore for users, organizations, sites, simulations, permissions, and roles. It also supported operational workflows and integrations.

S3-Compatible Storage

Object storage relied on an S3-compatible service for simulation assets, uploaded files, and project resources, but this storage was not fully integrated into a native AWS architecture.

GitBook Documentation Engine

GitBook was used as an external documentation platform for product documentation, deployment guides, technical manuals, and support material.

Notification Services

Operational notifications depended on external integrations such as Slack and Customer.io for account and workflow events.

5.1.5 Architectural Limitations

Although the initial architecture enabled rapid early development, several limitations emerged:

- strong dependency on Airtable as primary backend;
- limited scalability and weak cloud-native orchestration;
- high coupling with external services;
- manual operational workflows;
- increasing operational complexity over time.

These limitations motivated the migration path toward the AWS architecture presented in the next sections.

5.2 Architecture Overview

The migration of Outsight Cloud to AWS required the definition of a cloud-native architecture able to improve scalability, maintainability, and service integration while preserving the original application workflow.

The selected architecture follows a layered model with four levels:

- client and delivery layer;
- load balancing and routing layer;
- application layer;
- managed services layer.

Figure 5.2. Final AWS architecture adopted for Outsight Cloud.

Figure 5.2 shows the left-to-right request flow: users interact through the browser, traffic is delivered through CloudFront and routed by an Application Load Balancer, frontend and backend services run on ECS Fargate, and application logic integrates with managed services including PostgreSQL, Redis, Amazon S3, Cognito, SES, and SNS.

5.3 Client and Delivery Layer

5.3.1 Browser

The browser is the user entry point to Outsight Cloud. Through the frontend interface, users perform simulation visualization, organization and site management, document access, and file upload/download operations. All interactions occur over HTTPS.

5.3.2 Amazon CloudFront

Amazon CloudFront is used as the content delivery layer between users and the internal application infrastructure. Its role is to reduce latency, cache static assets, accelerate delivery, improve availability, and reduce backend load. This is particularly relevant for frontend static resources generated by the Vue application.

5.4 Load Balancing Layer

5.4.1 Application Load Balancer

The Application Load Balancer (ALB) is the central routing component. It receives requests forwarded by CloudFront and distributes traffic across services running on ECS Fargate. It provides request routing, health checks, HTTPS termination, and traffic distribution, enabling independent scaling and improved fault tolerance.

5.5 Application Layer

5.5.1 ECS Fargate Runtime

The application layer is deployed on Amazon ECS Fargate with a serverless container execution model. Compared to traditional VM-based deployment, Fargate removes the need to provision and maintain servers manually.

5.5.2 Frontend Service (Vue)

The frontend service is implemented in Vue.js and is responsible for rendering the user interface, handling user interactions, communicating with backend APIs, and managing authentication-related flows. Deploying it as a dedicated container enables independent release and scaling behavior.

5.5.3 Backend Service (Node.js API)

The backend service is implemented in Node.js and exposes the core APIs used by Outsight Cloud. It orchestrates business logic related to organizations, sites, simulations, authentication, file operations, notifications, and integrations with AWS managed services.

5.6 Managed Services Layer

5.6.1 PostgreSQL

PostgreSQL replaced Airtable as primary data storage. The migration objective was to move from a semi-structured operational backend toward a relational model with stronger consistency and integrity guarantees. Core entities such as users, organizations, sites, and simulations are persisted in PostgreSQL.

5.6.2 Redis

Redis is used as in-memory cache and session storage. It reduces repeated accesses to persistent storage and improves response time for frequently accessed information and temporary state management.

5.6.3 Amazon S3

Amazon S3 provides object storage for simulation files, uploaded assets, documentation artifacts, and deployment-related resources. It was selected for scalability, durability, and native integration with the AWS ecosystem.

5.6.4 Amazon Cognito

Amazon Cognito is used for authentication and identity management, including user registration, login workflows, identity verification, and token handling. This reduces custom authentication logic in the application layer.

5.6.5 Amazon SES

Amazon SES is used for outgoing email communication such as invitations, account-related messages, notifications, and operational communications.

5.6.6 Amazon SNS

Amazon SNS is adopted for asynchronous event-driven notifications. It supports decoupled communication patterns for alerts, internal notifications, workflow events, and service integration triggers.

5.7 Architectural Benefits

Compared to the pre-migration system, the selected architecture provides:

- clearer separation between presentation, business logic, and service layers;
- cloud-native deployment model with managed runtime and services;
- independent scaling of frontend and backend components;
- reduced operational burden through managed infrastructure;
- higher maintainability and better integration with IaC and automation workflows.

Overall, the final AWS design aligns with the broader cloudification strategy of Outsight Cloud and provides a robust baseline for future platform evolution.

Chapter 6

Migration Implementation

This chapter describes the implementation foundations that were already established to execute the migration strategy in a controlled and verifiable way.

6.1 Incremental Execution Plan

The migration was implemented incrementally, with explicit separation between foundation work and full functional cutover. The implementation sequence followed four tracks:

- establish cloud infrastructure foundations on AWS;
- formalize reproducible provisioning with Terraform;
- introduce data migration tooling from Airtable to PostgreSQL;
- implement validation pipelines to check behavioral consistency during coexistence.

This sequencing reduced migration risk by enabling technical validation before irreversible system decisions.

6.2 Infrastructure as Code and Environment Separation

6.2.1 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is a software engineering approach in which infrastructure is provisioned and managed through code instead of manual configuration procedures [2]. The rationale comes from DevOps principles: infrastructure should be treated with the same rigor applied to application code, including versioning, reviewability, traceability, and repeatability.

Under the IaC paradigm, infrastructure definitions are declarative artifacts stored in version control systems. This replaces fragile practices based on manual console operations, ad-hoc scripts, or outdated runbooks. As highlighted by AWS guidance, declarative infrastructure improves consistency and reliability in environment creation and updates [2].

A key IaC characteristic is that engineers define the desired final state, while automation tooling computes and executes the required operations. This model provides:

- **Consistency:** reproducible environments across development, testing, and production.
- **Version control:** auditable change history for infrastructure evolution.
- **Automation:** integration with CI/CD-driven deployment processes.
- **Reproducibility:** deterministic creation and update workflows.
- **Maintainability:** reusable infrastructure definitions that evolve over time.

In the Outsight Cloud migration, IaC was adopted to automate AWS provisioning and reduce manual intervention during environment setup and updates.

6.2.2 Terraform-Driven Provisioning

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp that enables provisioning and lifecycle management of infrastructure through declarative

configuration files [3]. Instead of manually creating resources through cloud consoles, infrastructure is defined as code in human-readable files (HCL), which can be versioned, reviewed, reused, and integrated into software delivery workflows.

Terraform interacts with platforms through providers, i.e., plugins that map Terraform resources to external APIs. In this migration, the AWS provider was used to manage cloud resources including load balancing, container runtime services, networking components, DNS/TLS integration, storage services, and observability-related infrastructure.

6.2.3 Terraform Workflow: Write, Plan, Apply

The Terraform workflow follows three phases [3]:

- **Write:** define the target infrastructure state declaratively in configuration files.
- **Plan:** generate an execution plan that compares desired state and current state, showing which resources will be created, updated, or removed.
- **Apply:** execute the planned operations while respecting resource dependencies through an internal dependency graph.

This workflow was especially useful for migration safety, because planned changes could be reviewed before execution and applied incrementally.

6.2.4 State Management and Team Collaboration

Terraform keeps a state representation of real infrastructure (`terraform.tfstate`) to compute incremental changes instead of recreating complete environments [3]. In practice, state management enabled drift detection and made environment updates predictable across migration iterations.

For collaborative operations, shared state and controlled execution are essential. This improves consistency across development stages and reduces configuration drift during parallel workstreams.

6.2.5 Why Terraform in the Outsight Cloud Migration

Terraform was selected because it aligned with the migration objectives:

- **Standardization:** infrastructure definitions became versioned artifacts.
- **Automation:** provisioning and updates were integrated into CI/CD-oriented processes.
- **Consistency:** environments were reproducible across development and production contexts.
- **Scalability:** infrastructure changes could be applied incrementally.
- **Maintainability:** modular and declarative definitions reduced manual intervention.

Within the Outsight Cloud project, this translated into a reproducible AWS infrastructure baseline supporting the broader cloudification strategy and reducing operational risk during migration.

6.2.6 Environment Governance

A dedicated effort was made to separate development and production concerns. This included environment-specific configuration handling and controlled evolution of shared resources to avoid production side effects during migration experimentation.

6.3 Data Migration Tooling and Validation

6.3.1 Relational Foundation

The migration path from Airtable to PostgreSQL was implemented with schema and migration tooling designed to preserve compatibility constraints. The objective was not only data transfer, but controlled evolution toward explicit schema ownership.

6.3.2 Replication and Validation Scripts

Dedicated scripts were prepared to:

- replicate data from Airtable into PostgreSQL;
- validate consistency through machine-readable reports;
- support migration checkpoints before wider rollout.

These artifacts transform data migration from a one-time operation into a repeatable verification workflow.

6.4 CI/Test Strategy for Dual-Backend Consistency

6.4.1 Dual-Backend Test Design

To support coexistence, smoke tests were designed to run against both data backends (Airtable and PostgreSQL) under a common API expectation. This provided an early warning mechanism for behavioral divergence.

6.4.2 Containerized Validation Environment

The test workflow uses containerized dependencies, including a PostgreSQL service and an Airtable-compatible mock component, so that migration checks can be reproduced reliably in CI.

6.4.3 Pipeline Integration

GitLab CI stages were defined to execute unit and smoke checks across backend variants. This allows migration decisions to be supported by continuous evidence instead of manual spot checks.

6.5 Implementation Scope and Current Status

At the current stage, the implemented work establishes the migration backbone:

- infrastructure foundations and environment governance;
- data replication and validation tooling;
- coexistence-oriented CI/test pipeline.

This scope is sufficient to evaluate migration feasibility and design quality, while leaving room for further functional expansion in subsequent project phases.

Chapter 7

Evaluation and Discussion

This chapter evaluates the migration approach with focus on decision quality, implementation readiness, and risk reduction evidence.

7.1 Validation Methodology

Given the current project stage, evaluation is organized around migration readiness rather than final production performance optimization. The methodology combines:

- architectural consistency checks between migration goals and implemented foundations;
- reproducibility checks through Infrastructure as Code and environment separation;
- coexistence checks through dual-backend testing and CI evidence;
- data migration checks through replication and validation scripts.

7.2 Technical Results

7.2.1 Infrastructure and Deployment Readiness

The migration produced a reproducible infrastructure baseline on AWS, managed through Terraform and integrated with routing, certificate management, container runtime, and observability services.

7.2.2 Data Migration Readiness

The project established concrete mechanisms to compare and validate behavior during Airtable-to-PostgreSQL transition. This is a key technical result because it enables incremental migration decisions based on evidence rather than assumptions.

7.2.3 Testing and Verification Readiness

Dual-backend smoke tests and CI integration provide an automated control surface for migration risk. This increases confidence in backward-compatible evolution during coexistence phases.

7.3 Migration KPIs and Economic Impact

The evaluation framework tracks migration progress using two KPI classes.

7.3.1 Technical Migration KPIs

- percentage of critical flows covered by backend-agnostic smoke tests;
- number of migration checkpoints completed with successful validation reports;
- reproducibility of environment provisioning across development stages;
- incidence of regressions detected before deployment.

7.3.2 Economic Migration KPIs

- reduction in dependency concentration on high-recurring-cost services;
- expected decrease in manual operational effort through IaC and pipeline automation;
- migration effort distribution across infrastructure, data, and application layers.

Initial evidence confirms that the selected strategy is compatible with both technical risk control and economic sustainability goals.

7.4 Limitations and Lessons Learned

7.4.1 Limitations

The current evaluation reflects an intermediate migration stage. Long-term operational behavior and full post-migration cost curves require additional observation after broader adoption.

7.4.2 Lessons Learned

Three lessons emerge clearly:

- migration architecture must be paired with explicit validation architecture;
- coexistence is manageable only when test and data checks are planned from the start;
- technical and economic criteria must be evaluated together, not sequentially.

Chapter 8

Conclusions and Future Work

This thesis addressed the migration of a legacy platform toward a cloud-native architecture as a structured engineering decision problem. The work focused on defining and justifying a migration strategy under technical and economic constraints.

8.1 Key Outcomes

The migration approach demonstrates that legacy-to-cloud-native transition can preserve continuity when it is treated as an incremental engineering process with explicit validation checkpoints.

The decision framework confirms the need to evaluate technical and economic criteria together. Security, compatibility, and operability decisions are sustainable only when recurring cost and operational effort are considered from the beginning.

The implementation evidence shows that coexistence can be managed through dual-backend testing, CI-enforced checks, and data validation artifacts. These elements make migration progress measurable and auditable.

Finally, the produced artifacts (decision rationale, Terraform definitions, migration scripts, validation workflows) improve reusability of the approach beyond a single project context.

8.2 Future Improvements

Future work can extend this thesis in three directions:

- complete and monitor full production cutover stages with long-term KPIs;
- further evolve identity architecture toward full standards-based integration across all services;
- consolidate post-migration cost and reliability analysis with extended operational data.

Overall, the thesis shows that cloud-native migration is most effective when treated as an explicit design process supported by verifiable checkpoints, rather than as a sequence of isolated implementation tasks.

Bibliography

[1] Amazon Web Services. «What is cloud migration?» Accessed: 2026-05-17. (2026), [Online]. Available: https://aws.amazon.com/what-is/cloud-migration/.

[2] Amazon Web Services. «Infrastructure as code». Accessed: 2026-05-17. (2026), [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/introduction-devops-aws/infrastructure-as-code.html.

[3] HashiCorp. «What is terraform?» Accessed: 2026-05-17. (2026), [Online]. Available: https://developer.hashicorp.com/terraform/intro.